

Application Design Document: Simple Two-User Chat Application

Team 3: Furqaanahmed Khan, Noel Erkie, Zheyu Han

Course: MSCS-632 Advanced Programming Languages

Instructor: Jay Thom

University: University of the Cumberland

Date: June 26, 2026

Project Overview

This project will implement a web-based chat application in two programming languages: Java and Go. The two implementations will provide the same user-facing behavior and follow the same application design. This is important because the final comparison should focus on language differences rather than differences in project scope. The application will allow users to enter a username, connect to the backend server, send messages to another user, view message history during the current server run, and search stored messages.

The system is intentionally small in scope. The goal is not to build a production chat platform. The goal is to create a working application that is simple enough to complete within the assignment timeline while still being meaningful for comparing Java and Go in areas such as concurrency, error handling, data modeling, modularity, and server-side programming style.

Goals

The application will use username-based access rather than full authentication. Users will enter a username before joining the chat, but the system will not use passwords, account verification, or secure login. This design keeps the project focused on message routing, in-memory storage, and language comparison.

In Scope

- Simple Sign-In: Users just type in a quick username to join. No passwords, accounts, or emails needed.
- Simulate Multiple Users: Let multiple people connect, start chats, and talk to each other at the exact same time.
- Smart History Tracking: Save messages automatically with the sender's name and the exact time it was sent.

- Easy Search: Let users quickly find old messages by searching for a specific keyword or username.
- Language Comparison: Use this setup to test and compare how different coding languages handle live chat traffic and temporary data storage.

Out of Scope

- No Permanent Saving: No databases. If the server restarts or crashes, all chat history and usernames are completely wiped out.
- No Group Chats: Conversations are strictly between two people. You cannot add a third person to a chat.
- No Advanced Security: No special encryption or hidden messaging.

Architecture Design

The application design centers on a lightweight two-user chat model. A user enters their own username and then finds the username of the person they want to chat with. Since every chat contains only two users, the system does not require a separate conversation table. Instead, the server can generate a deterministic conversation key from the two usernames. For example, a chat between `alice` and `bob` can be stored using a key such as `alice:bob`. The usernames should be sorted before creating the key so that `alice:bob` and `bob:alice` refer to the same message history.

Frontend Responsibilities

The frontend is intentionally basic and only serves to test the backend. It handles:

- Screens: Username login, recipient selection, live chat window, history logs, and a search bar.

- Actions: Capturing user input, opening the HTTP event stream (SSE) to listen for messages, sending HTTP requests, and displaying data or errors.

Backend Responsibilities

Both backend versions must handle the exact same core tasks:

- Stream Management: Open and maintain persistent HTTP event streams (SSE) for online users, and reject duplicate usernames.
- Routing: Generate a sorted chat key (e.g., alice + bob becomes alice:bob) to identify the 2-user chat room, then push live messages to the recipient's open event stream.
- Data & Search: Save messages to temporary memory and run basic keyword searches across them.

In-Memory Data Design

The application will use in-memory storage. This means users, active connections, and messages exist only while the server process is running. If the server restarts, the data will be lost. This is an intentional simplification because it avoids database setup and keeps the focus on the language comparison.

Data Structure	Purpose
users	Stores usernames known during the current server run.
ActiveConnections	Maps a username to the active WebSocket connection for that user.
messagesByConversation	Maps a two-user conversation key to a list of messages.

Data Model

The data model is small because the application does not require permanent database tables or complex relationships. The main models are a user session and a message.

Model	Field	Description
UserSession	username	The display name entered by the user.
UserSession	connected	Indicates whether the user currently has an active WebSocket connection.
UserSession	connectedAt	The time when the user connected to the server.
Message	messageId	A unique identifier for the message during the server run.
Message	sender	The username of the sender.
Message	recipient	The username of the intended recipient.
Message	text	The message content.
Message	timestamp	The time when the server received the message.

Error Handling Mechanism

- Duplicates: Rejects a login if the username is already active.
- Validation: Returns an error for empty messages or missing recipients without crashing.
- Offline Delivery: If a recipient is offline, the message is still saved to history but isn't delivered in real time.

Java and Go Implementation Alignment

Both implementations follow the same functional design, message format, and user-facing behavior, ensuring the final comparison reflects language differences rather than requirement differences.

Design Area	Java Implementation Direction	Go Implementation Direction
Data modeling	Classes (UserSession, Message)	Structs (UserSession, Message)
Shared state	ConcurrentHashMap or synchronized access to shared maps.	Maps protected by sync.Mutex or sync.RWMutex.
Connection handling	WebSocket handler classes and server framework support.	WebSocket handler functions and goroutines.
Concurrency	Threads, executors, or framework-managed request handling.	Goroutines with explicit synchronization.
Error handling	Exceptions and validation checks.	Explicit error return values and validation checks.
Modularity	Packages, classes, and interfaces.	Packages, structs, and functions.

Task Assignment

The table below provides a high-level plan for dividing the project work across the group. Since this is a small project with matching Java and Go implementations, all members should stay involved in each major area while still having a primary focus.

Area	Owner	Support	Main Deliverables
Design and Planning	All members	All members	Final application design, scope, assumptions, architecture, and shared protocol decisions
Java Implementation	All members	All members	Java backend with WebSocket handling, username sessions, message routing, in-memory history, and search
Go Implementation	All members	All members	Go backend with matching behavior to the Java version
Web Client and Integration	All members	All members	Simple web client connected to both backend versions for username entry, messaging, history, and search
Testing and Final Documentation	All members	All members	Shared test scenarios, debugging, README files, screenshots, and final Java vs. Go comparison

This task assignment keeps the work organized by major project area while showing that the group will work collaboratively across the full project. Everyone will contribute to each part of the project, including design, implementation, testing, and documentation. The group may still divide smaller day-to-day tasks based on availability and language familiarity, but all members will share the overall responsibility for each project area.

Project Timeline

The project timeline will be used to keep the Java and Go implementations aligned. Since both versions must provide the same core behavior, the group will first finalize the shared design and message protocol before starting the individual implementations. The timeline below is a working plan and can be updated based on the final assignment deadline.

Phase	Main Work Items	Target Date	Status
Design Finalization	Complete scope, architecture, application design, and task assignment	6/27/2026	In progress
Shared Protocol	Finalize WebSocket message types and common JSON structure	6/27/2026	Not started
Java Implementation	Build username handling, message routing, in-memory storage, history, and search	6/27/2026	Not started
Go Implementation	Build the same backend behavior using Go structs, maps, and concurrency tools	6/27/2026	Not started
Frontend Integration	Connect the web client to both backend versions	6/27/2026	Not started
Testing	Run the same test cases against the Java and Go versions	6/27/2026	Not started
Final Report	Compare Java and Go based on implementation differences	6/27/2026	Not started
Submission Review	Review formatting, README files, screenshots, and project packaging	6/27/2026	Not started

This timeline keeps the work organized around the main project requirement: building the same application in two programming languages. The most important checkpoint is the shared protocol phase, because both implementations need to use the same message types and expected behavior. Once the Java and Go versions are complete, testing should focus on confirming that both versions handle the same user actions, errors, message history, and search behavior.

Github Repo

https://github.com/hzy-stack/MSCS632_team3